МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность ПО-13

Выполнил:

Петручик Е.Н.

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

20.03.2026

Брест 2026

Цель работы:

Закрепить навыки объектно-ориентированного программирования на языке Python.

Задание 1

Реализовать простой класс

Требования к выполнению

• Реализовать пользовательский класс по варианту.

Для каждого класса

• Создать атрибуты (поля) классов

• Создать методы классов

• Добавить необходимые свойства и сеттеры (по необходимости)

• Переопределить магические методы __str__ и __eq__

7) Множество символов ограниченной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Мощность множества задается при создании объекта. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Код программы:

```
class CharSet:

    def __init__(self, max_size, initial=None):

        self._max_size = max_size

        self._items = []

        if initial:

            for ch in initial:

                self.add(ch)

    def max_size(self):

        return self._max_size

    def items(self):

        return self._items.copy()


    def add(self, ch):

        if len(ch) == 1 and ch not in self._items and len(self._items) <
self._max_size:

            self._items.append(ch)

            self._items.sort()
```

```
    def remove(self, ch):
        if ch in self._items:
            self._items.remove(ch)

    def contains(self, ch):
        return ch in self._items

    def union(self, other):
        result = CharSet(self._max_size + other._max_size)
        for ch in self._items:
            result.add(ch)
        for ch in other._items:
            result.add(ch)
        return result

    def display(self):
        print(self._items)

    def __contains__(self, ch):
        return ch in self._items

    def __str__(self):
        return f"CharSet({self._items}, max={self._max_size})"

    def __eq__(self, other):
        return sorted(self._items) == sorted(other._items)


if __name__ == "__main__":
    print("=== Демонстрация работы CharSet ===\n")

    s1 = CharSet(3, "abc")
    s2 = CharSet(2, "xy")

    print("Исходные множества:")
    print(f"s1: {s1}")
    print(f"s2: {s2}")
```

```
print("\nДобавление 'd' в s1:")
s1.add('d')
s1.display()

print("\nПопытка добавить 'z' в s2 (превышение мощности):")
s2.add('z')
s2.display()

print("\nУдаление 'b' из s1:")
s1.remove('b')
s1.display()

print(f"\nСодержит ли s1 'a'? {s1.contains('a')}")
print(f"Содержит ли s1 'b'? {'b' in s1}")

print("\nОбъединение s1 и s2:")
s3 = s1.union(s2)
s3.display()

s4 = CharSet(3, "ad")
print(f"\ns1: {s1}")
print(f"s4: {s4}")
print(f"s1 == s4? {s1 == s4}")
```

Спецификация ввода

Ввод не требуется. Программа сама проводит тестирование функциональности.

Рисунки с результатами работы программы

```
Демонстрация работы CharSet

Исходные множества:
s1: CharSet(['a', 'b', 'c'], max=3)
s2: CharSet(['x', 'y'], max=2)

Добавление 'd' в s1:
['a', 'b', 'c']

Попытка добавить 'z' в s2 (превышение мощности):
['x', 'y']

Удаление 'b' из s1:
['a', 'c']

Содержит ли s1 'a'? True
Содержит ли s1 'b'? False

Объединение s1 и s2:
['a', 'c', 'x', 'y']

s1: CharSet(['a', 'c'], max=3)
s4: CharSet(['a', 'd'], max=3)
s1 == s4? False
```

Задание 2

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы

7) Система Автобаза. Диспетчер распределяет заявки на Рейсы между Водителями и назначает для этого Автомобиль. Водитель может сделать заявку на ремонт. Диспетчер может отстранить Водителя от работы. Водитель делает отметку о выполнении Рейса и состоянии Автомобиля.

Код программы:

```
from datetime import datetime, timedelta

import random


class Vehicle:

    def __init__(self, vin, model, year, reg_number):

        self.vin = vin

        self.model = model

        self.year = year

        self.reg_number = reg_number

        self.is_working = True
```

```
        self.current_driver = None
        self.mileage = 0

    def __str__(self):
        status = "исправен" if self.is_working else "неисправен"
        driver = f", водитель: {self.current_driver.name}" if self.current_driver
else ""
        return f"{self.model} ({self.reg_number}), {status}{driver}"

    def __eq__(self, other):
        return isinstance(other, Vehicle) and self.vin == other.vin



class Truck(Vehicle):
    def __init__(self, vin, model, year, reg_number, capacity, has_trailer=False):
        super().__init__(vin, model, year, reg_number)
        self.capacity = capacity
        self.has_trailer = has_trailer

    def __str__(self):
        trailer = "с прицепом" if self.has_trailer else "без прицепа"
        return f"Грузовик {self.model} ({self.reg_number}), {self.capacity}т,
{trailer}"



class Bus(Vehicle):
    def __init__(self, vin, model, year, reg_number, seats, route_number=""):
        super().__init__(vin, model, year, reg_number)
        self.seats = seats
        self.route_number = route_number

    def __str__(self):
        return f"Автобус {self.model} ({self.reg_number}), {self.seats} мест,
маршрут: {self.route_number}"



class Driver:
    def __init__(self, driver_id, name, license_category, experience):
        self.id = driver_id
```

```
        self.name = name
        self.license_category = license_category
        self.experience = experience
        self.is_active = True
        self.current_vehicle = None
        self.trips = []
        self.repair_requests = []

    def __str__(self):
        status = "активен" if self.is_active else "отстранен"
        vehicle = f", авто: {self.current_vehicle.reg_number}" if
self.current_vehicle else ""
        return f"Водитель {self.name} (стаж: {self.experience} лет,
{status}{vehicle})"

    def apply_for_repair(self, description):
        if self.current_vehicle and not self.current_vehicle.is_working:
            request = RepairRequest(self, self.current_vehicle, description)
            self.repair_requests.append(request)
            print(f"{self.name}: заявка на ремонт
{self.current_vehicle.reg_number}")
            return request
        print(f"{self.name}: нет авто для ремонта")
        return None

    def complete_trip(self, trip, vehicle_status="исправен"):
        if trip in self.trips and trip.status == "в пути":
            trip.status = "выполнен"
            if vehicle_status == "неисправен":
                trip.vehicle.is_working = False
                print(f"{self.name}: рейс {trip.id} выполнен, авто неисправно")
            else:
                print(f"{self.name}: рейс {trip.id} выполнен, авто исправно")
            return True
        return False



class Trip:
    def __init__(self, trip_id, route, distance, priority="обычный"):
```

```
        self.id = trip_id
        self.route = route
        self.distance = distance
        self.priority = priority
        self.status = "ожидает"
        self.driver = None
        self.vehicle = None
        self.start_date = None
        self.end_date = None

    def __str__(self):
        return f"Рейс {self.id}: {self.route} ({self.distance}км), {self.status}"


class RepairRequest:
    def __init__(self, driver, vehicle, description):
        self.id = random.randint(1000, 9999)
        self.driver = driver
        self.vehicle = vehicle
        self.description = description
        self.date = datetime.now()
        self.is_approved = None

    def __str__(self):
        status = "ожидает"
        if self.is_approved is True:
            status = "одобрена"
        elif self.is_approved is False:
            status = "отклонена"
        return f"Заявка {self.id}: {self.vehicle.reg_number} - {status}"


class Dispatcher:
    def __init__(self, name, employee_id):
        self.name = name
        self.employee_id = employee_id
        self.assigned_trips = []
```

```
    def __str__(self):
        return f"Диспетчер {self.name}"

    def assign_trip(self, trip, driver, vehicle):
        if not driver.is_active:
            print(f"{self.name}: водитель {driver.name} отстранен")
            return False
        if not vehicle.is_working:
            print(f"{self.name}: авто {vehicle.reg_number} неисправно")
            return False
        if driver.current_vehicle:
            print(f"{self.name}: у водителя {driver.name} уже есть авто")
            return False

        trip.driver = driver
        trip.vehicle = vehicle
        trip.status = "назначен"
        driver.current_vehicle = vehicle
        vehicle.current_driver = driver
        self.assigned_trips.append(trip)
        print(f"{self.name}: рейс {trip.id} назначен {driver.name} на
{vehicle.reg_number}")
        return True

    def start_trip(self, trip):
        if trip.status == "назначен":
            trip.status = "в пути"
            trip.start_date = datetime.now()
            trip.driver.trips.append(trip)
            print(f"{self.name}: рейс {trip.id} начат")
            return True
        return False

    def suspend_driver(self, driver, reason=""):
        driver.is_active = False
        if driver.current_vehicle:
            driver.current_vehicle.current_driver = None
            driver.current_vehicle = None
```

```
        reason_text = f" по причине: {reason}" if reason else ""
        print(f"{self.name}: водитель {driver.name} отстранен{reason_text}")

    def process_repair_request(self, request, approve=True):
        request.is_approved = approve
        if approve:
            request.vehicle.is_working = False
            print(f"{self.name}: заявка {request.id} одобрена, авто
{request.vehicle.reg_number} в ремонт")
        else:
            print(f"{self.name}: заявка {request.id} отклонена")



class FleetManager(Dispatcher):
    def __init__(self, name, employee_id):
        super().__init__(name, employee_id)
        self.vehicles = []
        self.drivers = []

    def __str__(self):
        return f"Начальник автоколонны {self.name}"

    def add_vehicle(self, vehicle):
        self.vehicles.append(vehicle)
        print(f"{self.name}: добавлено авто {vehicle.reg_number}")

    def add_driver(self, driver):
        self.drivers.append(driver)
        print(f"{self.name}: добавлен водитель {driver.name}")

    def get_available_vehicles(self):
        return [v for v in self.vehicles if v.is_working and not v.current_driver]

    def get_available_drivers(self):
        return [d for d in self.drivers if d.is_active and not d.current_vehicle]

    def get_stats(self):
        print(f"\nСтатистика автобазы")
```

```
        print(f"Всего водителей: {len(self.drivers)}")
        print(f"Активных: {len([d for d in self.drivers if d.is_active])}")
        print(f"Всего авто: {len(self.vehicles)}")
        print(f"Исправных: {len([v for v in self.vehicles if v.is_working])}")
        print(f"В рейсе: {len([v for v in self.vehicles if v.current_driver])}")

print("АВТОБАЗА - СИСТЕМА УПРАВЛЕНИЯ")

manager = FleetManager("Сидоров П.А.", "MGR001")
print(f"\nСоздан {manager}")

print("\nДобавление транспорта")
truck1 = Truck("VIN001", "MAN", 2021, "A123BB", 20, True)
truck2 = Truck("VIN002", "Scania", 2020, "B456CC", 25, False)
bus1 = Bus("VIN003", "ЛиАЗ", 2022, "К789ММ", 50, "101")
bus2 = Bus("VIN004", "МАЗ", 2021, "О321РР", 45, "205")

manager.add_vehicle(truck1)
manager.add_vehicle(truck2)
manager.add_vehicle(bus1)
manager.add_vehicle(bus2)

print("\nДобавление водителей")
driver1 = Driver(1, "Иванов И.И.", "CE", 12)
driver2 = Driver(2, "Петров П.П.", "CE", 8)
driver3 = Driver(3, "Сидоров С.С.", "D", 15)
driver4 = Driver(4, "Козлов А.А.", "D", 5)

manager.add_driver(driver1)
manager.add_driver(driver2)
manager.add_driver(driver3)
manager.add_driver(driver4)

print("\nСоздание диспетчера")
dispatcher = Dispatcher("Иванова М.И.", "DSP001")
print(dispatcher)

print("\nСоздание рейсов")
```

```
trip1 = Trip(101, "Минск – Брест", 350, "срочный")
trip2 = Trip(102, "Минск – Гродно", 300, "обычный")
trip3 = Trip(103, "Минск – Витебск", 280, "обычный")
trip4 = Trip(104, "Минск – Гомель", 320, "срочный")

print(f"Созданы рейсы: {trip1.id}, {trip2.id}, {trip3.id}, {trip4.id}")

print("\nНазначение рейсов")
print("Доступные водители:", [d.name for d in manager.get_available_drivers()])
print("Доступные авто:", [v.reg_number for v in manager.get_available_vehicles()])

print("\nНазначение рейсов:")
dispatcher.assign_trip(trip1, driver1, truck1)
dispatcher.assign_trip(trip2, driver2, truck2)
dispatcher.assign_trip(trip3, driver3, bus1)

print("\nНачало рейсов:")
dispatcher.start_trip(trip1)
dispatcher.start_trip(trip2)

print("\nПопытка назначить занятого водителя")
dispatcher.assign_trip(trip4, driver1, bus2)

print("\nЗавершение рейса с неисправностью")
driver1.complete_trip(trip1, "неисправен")
print(f"Статус авто {truck1.reg_number}: {'исправен' if truck1.is_working else
'неисправен'}")

print("\nЗаявка на ремонт")
request = driver1.apply_for_repair("Замена тормозных колодок")
dispatcher.process_repair_request(request, approve=True)
print(f"Статус авто после ремонта: {'исправен' if truck1.is_working else
'неисправен'}")

print("\nОтстранение водителя")
dispatcher.suspend_driver(driver2, "нарушение графика")
print(driver2)
```

```
print("\nПопытка назначить отстранённого водителя")
dispatcher.assign_trip(trip4, driver2, bus2)

print("\nЗавершение второго рейса")
driver2.complete_trip(trip2)

print("\nДоступные после рейсов")
print("Доступные водители:", [d.name for d in manager.get_available_drivers()])
print("Доступные авто:", [v.reg_number for v in manager.get_available_vehicles()])

print("\nНазначение оставшихся рейсов")
dispatcher.assign_trip(trip3, driver4, bus2)
dispatcher.assign_trip(trip4, driver3, bus1)
dispatcher.start_trip(trip3)
dispatcher.start_trip(trip4)

print("\nЗавершение всех рейсов")
driver3.complete_trip(trip3)
driver4.complete_trip(trip4)
driver1.complete_trip(trip4)

manager.get_stats()

print("\nИтоговое состояние")
print(f"Водители:")
for d in manager.drivers:
    print(f"  {d}")

print(f"\nТранспорт:")
for v in manager.vehicles:
    print(f"  {v}")
```

Спецификация ввода

Ввод не требуется. Программа сама проводит тестирование функциональности.

Рисунки с результатами работы программы

```
АВТОБАЗА - СИСТЕМА УПРАВЛЕНИЯ

Создан Начальник автоколонны Сидоров П.А.

Добавление транспорта
Сидоров П.А.: добавлено авто А123ВВ
Сидоров П.А.: добавлено авто В456СС
Сидоров П.А.: добавлено авто К789ММ
Сидоров П.А.: добавлено авто О321РР

Добавление водителей
Сидоров П.А.: добавлен водитель Иванов И.И.
Сидоров П.А.: добавлен водитель Петров П.П.
Сидоров П.А.: добавлен водитель Сидоров С.С.
Сидоров П.А.: добавлен водитель Козлов А.А.

Создание диспетчера
Диспетчер Иванова М.И.

Создание рейсов
Созданы рейсы: 101, 102, 103, 104

Назначение рейсов
Доступные водители: ['Иванов И.И.', 'Петров П.П.', 'Сидоров С.С.', 'Козлов А.А.']
Доступные авто: ['А123ВВ', 'В456СС', 'К789ММ', 'О321РР']

Назначение рейсов:
Иванова М.И.: рейс 101 назначен Иванов И.И. на А123ВВ
Иванова М.И.: рейс 102 назначен Петров П.П. на В456СС
Иванова М.И.: рейс 103 назначен Сидоров С.С. на К789ММ

Начало рейсов:
Иванова М.И.: рейс 101 начат
Иванова М.И.: рейс 102 начат
```

```
Попытка назначить занятого водителя
Иванова М.И.: у водителя Иванов И.И. уже есть авто

Завершение рейса с неисправностью
Иванов И.И.: рейс 101 выполнен, авто неисправно
Статус авто А123ВВ: неисправен

Заявка на ремонт
Иванов И.И.: заявка на ремонт А123ВВ
Иванова М.И.: заявка 5980 одобрена, авто А123ВВ в ремонт
Статус авто после ремонта: неисправен

Отстранение водителя
Иванова М.И.: водитель Петров П.П. отстранен по причине: нарушение графика
Водитель Петров П.П. (стаж: 8 лет, отстранен)

Попытка назначить отстранённого водителя
Иванова М.И.: водитель Петров П.П. отстранен

Завершение второго рейса
Петров П.П.: рейс 102 выполнен, авто исправно

Доступные после рейсов
Доступные водители: ['Козлов А.А.']
Доступные авто: ['В456СС', 'О321РР']

Назначение оставшихся рейсов
Иванова М.И.: рейс 103 назначен Козлов А.А. на О321РР
Иванова М.И.: у водителя Сидоров С.С. уже есть авто
Иванова М.И.: рейс 103 начат

Завершение всех рейсов
```

Вывод:

Я закрепил навыки объектно-ориентированного программирования на языке Python.